

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 9
REFACTORING**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	1
BAB I. HASIL PRAKTIKUM	2
I.1 Refactoring Class	2
I.1.A. Source Code	2
I.1.B. Analisa	3
I.2 Conditional	3
I.2.A. Source code	3
I.2.B. Analisa	5
I.3 Contoh Kasus	5
I.3.A. Source code	5
I.3.B. Analisa	9
BAB II. TUGAS PRAKTIKUM	10
II.1 Writing Code	10
II.1.A. Source code	10
II.1.B. Analisis	11
BAB III. KESIMPULAN	12

BAB I. HASIL PRAKTIKUM

I.1 Refactoring Class

I.1.A. Source Code

Person.java

```
package Latihan.RefactoringClass;

public class Person {
    public String name;
}
```

personRefactoring.java

```
package Latihan.RefactoringClass;

public class personRefactoring {
    public String name;
    public String getName() {
        return name;
    }

    public void setName(String newName) {
        name = newName;
    }
}
```

Main.java

```
package Latihan.RefactoringClass;

public class Main {
    public static void main(String[] args) {
        Person person = new Person();
        person.name = "Bobo Smith";
        System.out.println(person.name);

        personRefactoring personRefactoring = new personRefactoring();
        personRefactoring.setName("Bobob Smith");
    }
}
```

```
        System.out.println(personRefactoring.name);
    }
}
```

I.1.B. Analisa

Pada praktikum modul refactoring, kode di atas menunjukkan perbandingan antara kelas yang belum direfaktori (Person) dan kelas yang sudah direfaktori (personRefactoring). Pada kelas Person, atribut name bersifat public sehingga dapat diakses dan diubah secara langsung dari kelas Main. Hal ini terlihat pada penulisan `person.name = "Bobo Smith"`; yang menyebabkan kurangnya kontrol terhadap perubahan data. Selain itu, akses langsung terhadap atribut berpotensi menimbulkan kesalahan dan melanggar prinsip encapsulation dalam pemrograman berorientasi objek. Proses refactoring dilakukan pada kelas `personRefactoring` dengan menambahkan metode getter dan setter, yaitu `getName()` dan `setName()`. Dengan pendekatan ini, pengubahan nilai atribut name dilakukan melalui metode `setName()`, sehingga struktur kode menjadi lebih rapi, terkontrol, dan mudah dikembangkan. Refactoring ini meningkatkan kualitas desain program karena menerapkan prinsip encapsulation, memudahkan pemeliharaan kode, serta mengurangi risiko kesalahan saat atribut diakses atau dimodifikasi di masa depan.

I.2 Conditional

I.2.A. Source code

Sebelum

```
package Latihan.Conditional;

public class Main {
    public static void main(String[] args) {
        int score = 800;
        int income = 100000;
        boolean authorized = true;

        if (score > 700) accept();
        else if ((income >= 40000) && (income <= 100000))
```

```

        && authorized && (score > 500)) accept();
    else if (income > 100000) accept();
    else reject();
}

public static void accept() {
    System.out.println("You are accepted!");
}

public static void reject() {
    System.out.println("You are rejected!");
}
}

```

Sesudah

```

package Latihan.Conditional;

public class Main {

    int score = 800;
    int income = 100000;
    boolean authorized = true;

    if ((score <= 700) && ((income < 40000) || (income > 100000)
|| !authorized || (score <= 500)) &&(income <= 100000))
    {
        reject();
    }else {
        accept();
    }

    public static void accept() {
        System.out.println("You are accepted!");
    }

    public static void reject() {
        System.out.println("You are rejected!");
    }
}

```

```
}
```

I.2.B. Analisa

Pada kode sebelum refactoring, logika penerimaan dan penolakan ditulis menggunakan beberapa kondisi if-else if-else yang terpisah. Setiap kondisi memeriksa kombinasi nilai score, income, dan authorized untuk menentukan apakah metode accept() atau reject() dipanggil. Walaupun logika ini masih dapat dipahami, struktur percabangan yang panjang membuat kode menjadi kurang ringkas dan berpotensi sulit dipelihara apabila terdapat penambahan atau perubahan aturan di kemudian hari. Selain itu, adanya beberapa pemanggilan accept() di cabang yang berbeda menunjukkan adanya redundansi logika.

Pada kode sesudah refactoring, logika kondisi disederhanakan dengan memusatkan fokus pada kondisi penolakan (reject) terlebih dahulu, sedangkan kondisi lainnya secara implisit dianggap sebagai penerimaan (accept). Pendekatan ini membuat struktur kode lebih singkat dan mudah dibaca karena hanya menggunakan satu percabangan utama if-else. Refactoring ini bertujuan meningkatkan keterbacaan (readability) dan efisiensi kode tanpa mengubah perilaku program, sehingga logika bisnis tetap sama tetapi lebih terstruktur dan mudah dipahami dalam konteks pemeliharaan program.

I.3 Contoh Kasus

I.3.A. Source code

Sebelum

```
package Latihan.Case.Sebelum;

public class Validation {
    private boolean
        ValidateRequest(CustomerInquiryRequest request, int
customerIDFieldLength, int productFieldLength) {
        if (request.CustomerProduct.ProductNumber != null &&
request.Customer.CustomerID != null) {
            if (request.CustomerProduct.ProductNumber != string.Empty
&& request.Customer.CustomerID != string.Empty) {
                // Both were populated
                throw new
```

```

BusinessException(HandledErrors.InvalidBothParameterMessage);
        } else if (request.Customer.CustomerID == string.Empty &&
request.CustomerProduct.ProductNumber == string.Empty) {
// if objects were instantiated but not populated
        throw new
BusinessException(HandledErrors.CustomerEmptyMessage);
        } else if (request.Customer.CustomerID != "") {
// Note: ProductNumber was equal
// to string.empty Check Customer ID length
        if (request.Customer.CustomerID.Length >
customerIDFieldLength) {
            throw new
BusinessException(HandledErrors.CustomerInvalidLengthMessage);
        }
        } else {
// Note: CustomerID was equal
// to string.empty check Product Length
        if (request.CustomerProduct.ProductNumber.Length >
productFieldLength) {
            throw new
BusinessException(HandledErrors.ProductInvalidLengthMessage);
        }
    }

    } else if (request.CustomerProduct.ProductNumber == null &&
request.Customer.CustomerID == null) {
// Both were null
        throw new
BusinessException(HandledErrors.CustomerEmptyMessage);
    } else if (request.CustomerProduct.ProductNumber == null) {
// ProductNumber was null and CustomerID was not null
        if (request.Customer.CustomerID.Length >
customerIDFieldLength) {
            throw new
BusinessException(HandledErrors.CustomerInvalidLengthMessage);
        }
    } else { // ProductNumber not null and CustomerID was null
// Check Product Length
        if (request.CustomerProduct.ProductNumber.Length >
productFieldLength) {
            throw new

```

```

        BusinessException(HandledErrors.ProductInvalidLengthMessage);
    }
}

// Set objects below with formatted data i.e PadLeft
    if (request.Customer.CustomerID != null) {
        request.Customer.CustomerID =
request.Customer.CustomerID.PadLeft(customerIDFieldLength,
Convert.ToChar("0", CultureInfo.CurrentCulture));

    }
    if (request.CustomerProduct.ProductNumber != null) {
        request.CustomerProduct.ProductNumber =
request.CustomerProduct.ProductNumber.PadLeft(productFieldLength,
Convert.ToChar("0", CultureInfo.CurrentCulture));
    }
    return true;
}
}

```

Sesudah

```

package Latihan.Case.Sesudah;

public class Validation {
    private void ValidateRequest(CustomerInquiryRequest request, int
customerFieldLength, int productFieldLength) {
// 1. Check both parameters are not null and not empty strings
        CheckCustomerInquiryNotNullOrEmpty(request);
// 2. Check both parameters aren't populated
        CheckCustomerInquiryNullOrEmpty(request);
// 3. Check CustomerID for field length and pad the parameter
        CheckCustomerIDValid(request, customerFieldLength);
// 4. Check ProductNumber for field length and pad the parameter
        CheckProductNumberValid(request, productFieldLength);

    }

    private void
CheckCustomerInquiryNotNullOrEmpty(CustomerInquiryRequest request) {

```



```

// Check both parameters are not null or empty string
    if (!string.IsNullOrEmpty(request.CustomerProduct.ProductNumber)
    && !string.IsNullOrEmpty(request.Customer.CustomerID)) {
// Both were populated
        throw new
BusinessException(HandleErrors.InvalidBothParameterMessage);
    }
}

    private void CheckCustomerInquiryNullOrEmpty(CustomerInquiryRequest
        request) {
        if (string.IsNullOrEmpty(request.Customer.CustomerID) &&
string.IsNullOrEmpty(request.CustomerProduct.ProductNumber)) {
// Both are null or empty string
            throw new
BusinessException(HandleErrors.CustomerEmptyMessage);
        }
    }

    private void CheckCustomerIDValid(CustomerInquiryRequest request,
int customerIDFieldLength) {
        if (string.IsNullOrEmpty(request.Customer.CustomerID)) {
// Check Customer ID length
            if (request.Customer.CustomerID.Length >
customerIDFieldLength) {
                throw new
BusinessException(HandleErrors.CustomerInvalidLengthMessage);
            }
// Pad the left of the customer id
            request.Customer.CustomerID =
request.Customer.CustomerID.PadLeft(customerIDFieldLength,
Convert.ToChar("0", CultureInfo.CurrentCulture));
        }
    }

    private void CheckProductNumberValid(CustomerInquiryRequest request,
int productFieldLength) {
        if (string.IsNullOrEmpty(request.CustomerProduct.ProductNumber))
{
// Check Product Length
            if (request.CustomerProduct.ProductNumber.Length >

```

```

productFieldLength) {
    throw new
    BusinessException(HandledErrors.ProductInvalidLengthMessage);
}

// Pad the left of the product number
    request.CustomerProduct.ProductNumber =
    request.CustomerProduct.ProductNumber.PadLeft(productFieldLength,
    Convert.ToChar("0", CultureInfo.CurrentCulture));
}
}
}

```

I.3.B. Analisa

Pada kode sebelum refactoring, metode `ValidateRequest` memiliki struktur logika yang sangat kompleks dengan banyak percabangan `if-else` yang saling bertumpuk. Kondisi pengecekan nilai `null`, string kosong, validasi panjang field, serta proses padding dilakukan dalam satu metode yang sama. Hal ini menyebabkan kode sulit dibaca, sulit dipahami alurnya, dan berpotensi menimbulkan kesalahan ketika dilakukan perubahan. Selain itu, metode ini melanggar prinsip `Single Responsibility` karena satu metode menangani terlalu banyak tugas sekaligus.

Pada kode sesudah refactoring, logika validasi dipecah menjadi beberapa metode kecil yang masing-masing memiliki tanggung jawab spesifik, seperti `CheckCustomerInquiryNotNullOrEmpty`, `CheckCustomerInquiryNullOrEmpty`, `CheckCustomerIDValid`, dan `CheckProductNumberValid`. Pendekatan ini membuat kode lebih terstruktur, mudah dibaca, dan mudah dipelihara. Refactoring ini meningkatkan kualitas desain perangkat lunak dengan menerapkan prinsip `clean code` dan `separation of concerns`, sehingga setiap aturan validasi menjadi jelas, terisolasi, dan lebih mudah diuji maupun dikembangkan di masa mendatang.

BAB II. TUGAS PRAKTIKUM

II.1 Writing Code

II.1.A. Source code

Sebelum

```
package Tugas.Sebelum;

import java.io.File;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Group {
    // Gets users sorted by the most recently registered user
    public List getUsers() {
        List users = new ArrayList();
        if (!new File(persistencePath()).exists()) return users;
        File[] files = new File(persistencePath()).listFiles();
        for (File file : files) if (file.isDirectory()) users.add(new
        User(file.getName(), this));
        Collections.sort(users, new
        User.UserComparatorByDescendingRegistration());
        return users;
    }
}
```

Sesudah

```
package Tugas.Sesudah;

import java.util.ArrayList;
import java.util.List;

public class Group {
    public List getUsersSortedByMostRecentlyRegistered()
    {
        List users = new ArrayList();
        if (!userDirectoryExists()) return users;
        addFoundUsersTo(users);
    }
}
```

```

        sortByMostRecentlyRegistered(users);
        return users;
    }

    private void sortByMostRecentlyRegistered(List users) {

    }

    private void addFoundUsersTo(List users) {

    }

    private boolean userDirectoryExists() {
        return false;
    }
}

```

II.1.B. Analisis

Pada kode sebelum refactoring, metode `getUsers()` menangani beberapa tanggung jawab sekaligus, yaitu mengecek keberadaan direktori penyimpanan, membaca data pengguna dari sistem berkas, menambahkan pengguna ke dalam daftar, serta melakukan proses pengurutan. Seluruh proses tersebut ditulis dalam satu metode sehingga logika menjadi padat dan kurang jelas. Kondisi ini menyulitkan pembacaan kode dan melanggar prinsip Single Responsibility, karena satu metode seharusnya hanya memiliki satu tujuan utama.

Pada kode sesudah refactoring, metode utama diubah menjadi `getUsersSortedByMostRecentlyRegistered()` dengan nama yang lebih deskriptif dan jelas mencerminkan fungsinya. Proses-proses penting dipecah ke dalam beberapa metode terpisah seperti `userDirectoryExists()`, `addFoundUsersTo()`, dan `sortByMostRecentlyRegistered()`. Pemisahan ini membuat alur program lebih mudah dipahami, meningkatkan keterbacaan kode, serta memudahkan proses pemeliharaan dan pengembangan di masa depan. Refactoring ini menekankan penerapan prinsip clean code dan desain yang lebih modular tanpa mengubah fungsi utama program.

BAB III. KESIMPULAN

Berdasarkan hasil praktikum modul refactoring, dapat disimpulkan bahwa refactoring merupakan proses penting dalam pengembangan perangkat lunak untuk meningkatkan kualitas struktur kode tanpa mengubah perilaku atau fungsi program. Melalui beberapa studi kasus, terlihat bahwa kode awal umumnya masih memiliki masalah seperti struktur yang kompleks, percabangan berlebihan, penamaan metode yang kurang jelas, serta pelanggaran prinsip encapsulation dan Single Responsibility. Kondisi tersebut dapat menyulitkan proses pembacaan dan pemeliharaan kode.

Setelah dilakukan refactoring, kode menjadi lebih terstruktur, modular, dan mudah dipahami. Pemisahan logika ke dalam metode-metode kecil dengan tanggung jawab yang jelas, penggunaan nama metode yang deskriptif, serta penerapan prinsip object-oriented programming dan clean code terbukti mampu meningkatkan keterbacaan dan kejelasan alur program. Selain itu, refactoring juga mengurangi redundansi dan mempermudah pengujian serta pengembangan lanjutan.

Dengan demikian, praktikum ini memberikan pemahaman bahwa refactoring bukan hanya sekadar memperindah kode, tetapi merupakan langkah strategis untuk meningkatkan kualitas dan keberlanjutan perangkat lunak. Penerapan refactoring secara konsisten akan membantu pengembang dalam mengelola kode yang lebih aman, fleksibel, dan mudah dirawat, terutama pada proyek berskala besar atau jangka panjang.